

TRANSPORT LAYER

The transport layer is located between the application layer and the network layer. It provides a process-to-process communication between two application layers, one at the local host and the other at the remote host.

TRANSPORT LAYER SERVICES:

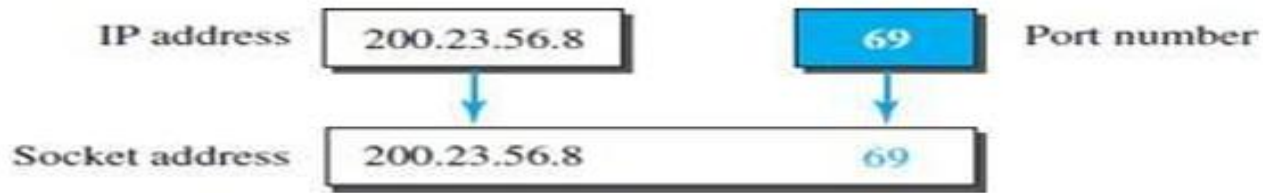
- ✓ **PROCESS TO PROCESS COMMUNICATION** :The **Transport Layer** solves this problem by delivering the received message to the **appropriate process** using **port numbers**. Each application is assigned a unique port number, which helps the transport layer identify the correct destination process.

Example: Web browser → Port 80 (HTTP) Email → Port 25 (SMTP)

✓ Addressing: Port Numbers

- **Port numbers** identify specific processes/applications on a host.
- Communication requires:
 - **IP Address** → Identifies the host (computer)
 - **Port Number** → Identifies the process/application
- Port numbers are **16-bit integers** ranging from **0 to 65,535**.
- **Client applications** use **Ephemeral Port Numbers** (temporary ports, usually > 1023).
- **Server applications** use **Well-Known Port Numbers** (fixed, standardized ports).
- Clients know the well-known port number of the server they want to communicate with.
- **Examples:**
 - HTTP → Port 80
 - HTTPS → Port 443
 - FTP → Port 21
 - SMTP → Port 25

✓ Socket Addresses



✓ Encapsulation and Decapsulation

Encapsulation (Sender Side)

- ❖ Application process sends data to the Transport Layer.
- ❖ Transport Layer adds a **transport header** to the data.
- ❖ The resulting unit is called a **segment**, **user datagram**, or **packet** (depending on the protocol).
- ❖ Data is then forwarded to the lower layers for transmission.

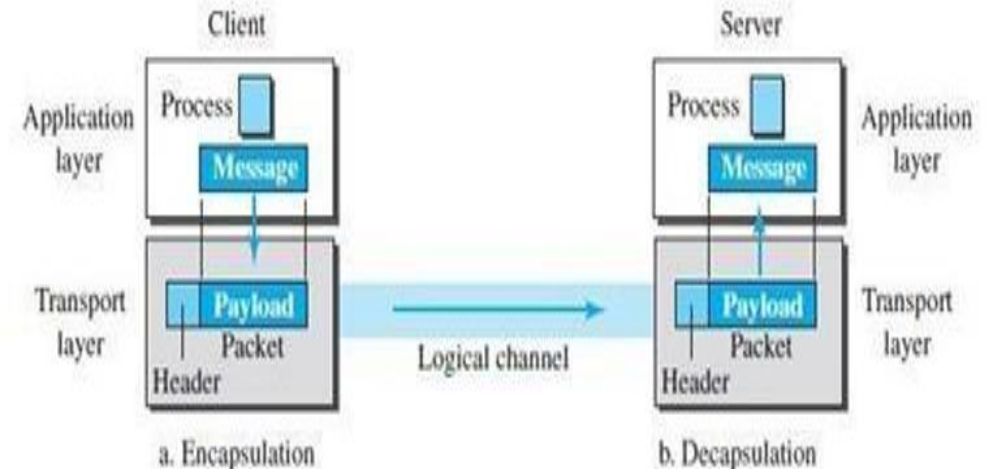
Decapsulation (Receiver Side)

- ❖ Transport Layer receives the packet.
- ❖ Removes the **transport header**.
- ❖ Delivers the original message to the correct application process.
- ❖ Sender's socket address is provided if a reply is needed.

Key Point:

Encapsulation = Adding headers at the sender.

Decapsulation = Removing headers at the receiver.



✓ Multiplexing and Demultiplexing

Multiplexing (Many → One)

Performed at the **sender side**.

Combines data from **multiple applications/processes** into a single stream for transmission.

Demultiplexing (One → Many)

Performed at the **receiver side**.

Delivers received data to the **correct application/process** using port numbers.

✓ Pushing and Pulling

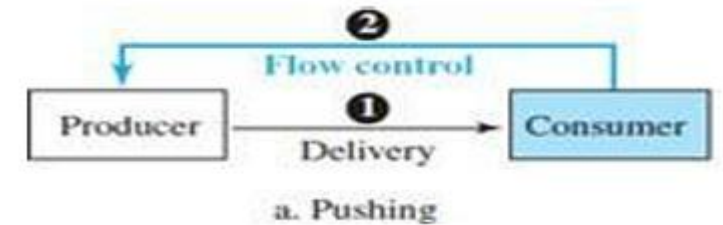
Pushing

The **sender (producer)** sends data whenever it is ready.

The **receiver (consumer)** may become overloaded if data arrives too quickly.

Therefore, **flow control** is required to regulate transmission.

Example: TCP data transfer, where the receiver controls the sender using window size information.



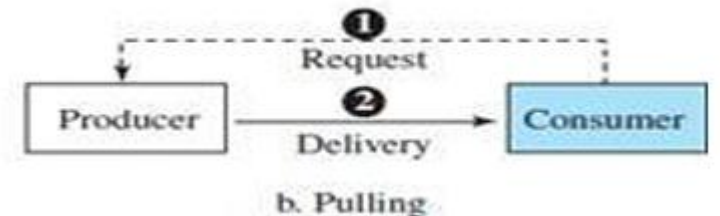
Pulling

The **receiver (consumer)** requests data only when it is ready.

Data is sent only upon request.

No flow control is needed because the receiver controls the rate of transfer.

Example: Downloading a webpage, where the browser requests data from the server.



✓ Flow Control at the Transport Layer

- **Flow control** prevents the receiver from being overwhelmed by excessive data.
- Four entities are involved:
 - **Sender Process (Application Layer)** – Produces and sends data.
 - **Sender Transport Layer** – Receives data, encapsulates it into packets, and forwards it.
 - **Receiver Transport Layer** – Receives packets, decapsulates them, and stores the data.
 - **Receiver Process (Application Layer)** – Requests and receives the data.



✓ Buffers in Transport Layer Flow Control

What is a Buffer?

A **buffer** is a temporary memory area used to store data packets during transmission.

Two buffers are commonly used:

Sender Buffer – Stores outgoing packets.

Receiver Buffer – Stores incoming packets.

How Flow Control Works?

If the **sender buffer is full**, it tells the application to **pause sending data**.

If the **receiver buffer is full**, it tells the sender to **stop transmitting packets**.

When space becomes available, communication resumes.

✓ **Error Control in the Transport Layer**

▪ **Purpose**

Since the Network Layer (IP) is unreliable, the Transport Layer provides error control to ensure reliable data delivery.

▪ **Responsibilities of Error Control**

Detects and discards corrupted packets.

Identifies lost packets and requests retransmission.

Recognizes and removes duplicate packets.

Buffers out-of-order packets and rearranges them before delivery.

✓ Sequence Numbers in Transport Layer

What are Sequence Numbers?

Each packet is assigned a **sequence number** to identify its order during transmission.

Sequence numbers help in:

- ❖ Detecting lost packets
- ❖ Identifying duplicate packets
- ❖ Reordering out-of-sequence packets

Sequence Number Range

If **m bits** are allocated for the sequence number:

Range = 0 to $(2^m - 1)$

Example: **m = 4 bits**

- RANGE = ?

✓ Sliding Window (Transport Layer)

What is a Sliding Window?

A **Sliding Window** is a flow control mechanism used by TCP.

It allows the sender to transmit multiple packets before receiving acknowledgments.

The window represents the packets that can be sent and stored in the buffer.

How It Works

Packets are assigned **sequence numbers**.

When a packet is sent, its slot in the window is marked.

If all slots are occupied, the sender must wait for acknowledgments.

When an acknowledgment (ACK) is received, the corresponding slot is freed.

The window **slides forward**, allowing new packets to be sent.

✓ Connectionless and Connection-Oriented Protocols

